紫紅底

數位邏輯設計實驗結報

- 實驗班別：■資工(星期三)
- 實驗名稱：(五) 4位元減法器
- 實驗日期：2025 年____月____日
- 組別編號：22
- 成員名單：

【實驗目的】

利用 4-bit 加法器完成 4-bit 減法器。

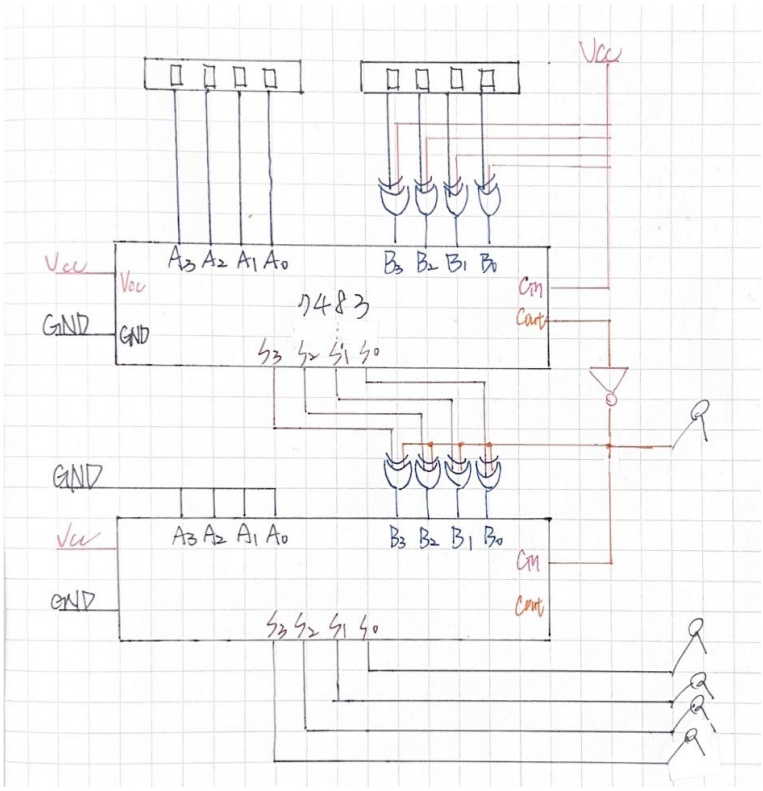
【實驗原理】

在四位元計算中，A － B可以用 A + B 的 2's 補數完成。

若 A ＞ B，則 A + B 的 2's 補數**會進位**，忽略進位餘下的值即為結果，值為**正**。

若 A＜B，則 A＋B 的 2's 補數**不會進位**，值不正確，需將值**再做一次 2's 補數**，使其值正確，並為**負**。

【電路設計】



電路圖簡化

1. **XOR 轉換 B3-B0**（產生 1 的補數）
   - 我們對 **B3-B0** 的每一位元與 **VCC（1）** 進行 XOR 運算（7486IC），這會將 B3-B0 轉換為 **一的補數**：

$$B_i' = B_i \oplus 1 = \overline{B_i}$$

B3-B0 值	VCC（1）	XOR 運算結果	功用
若為 0	1	1	將 0 反轉
若為 1	1	0	將 1 反轉

   - 準備進行二的補數運算。

2. **第一個 7483：計算 A - B = A + B 二的補數**
   - 在 **第一個 7483 的 Cin 輸入 VCC（1）**，使得 B3-B0 進一步變為 **二的補數**：

$$A - B = A + (\overline{B} + 1)$$

   - **輸出 S3-S0** 是第一組 4 位元的減法結果。
   - **Cout 反映進位狀況**，表示是否需要再進行一次二的補數。

3. **處理 Cout（負數偵測）**
   - **第一個 7483 的 Cout 代表進位情況**：
     - Cout = 1：代表 A ≥ B，結果為正數或零。
     - Cout = 0：代表 A < B，結果為負數。
   - 透過一個 **INVERSE**（7404IC）反轉 Cout：

$$Cout' = \overline{Cout}$$

     - **負數時 Cout' = 1**
     - **正數時 Cout' = 0**
   - 這個反轉後的 Cout' 會參與第二個 7483 的運算。
   - 將 Cout' 並聯一個 led 燈，燈不亮時為正，燈亮時為負。

4. **第一個 7483 的輸出 S3-S0 與 Cout' 進行 XOR**（7486IC）：
   - 如果結果是正數（Cout' = 0），則 S3-S0 保持不變。

S3-S0 值	Cout' = 0	XOR 運算結果	功用
若為 0	0	0	值不變
若為 1	0	1	

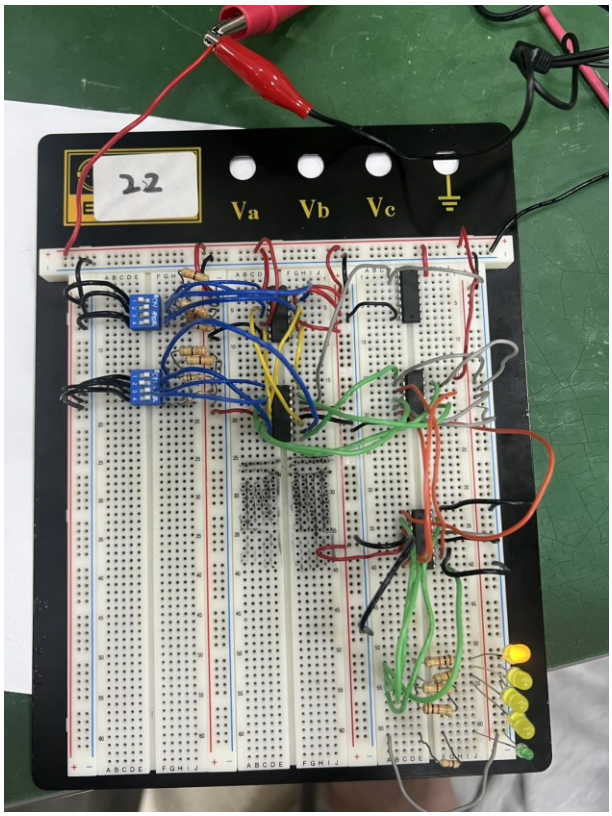
   - 如果結果是負數（Cout' = 1），則對 S3-S0 取補數，使其成為一的補數。

S3-S0 值	Cout' = 1	XOR 運算結果	功用
若為 0	1	1	將 0 反轉
若為 1	1	0	將 1 反轉

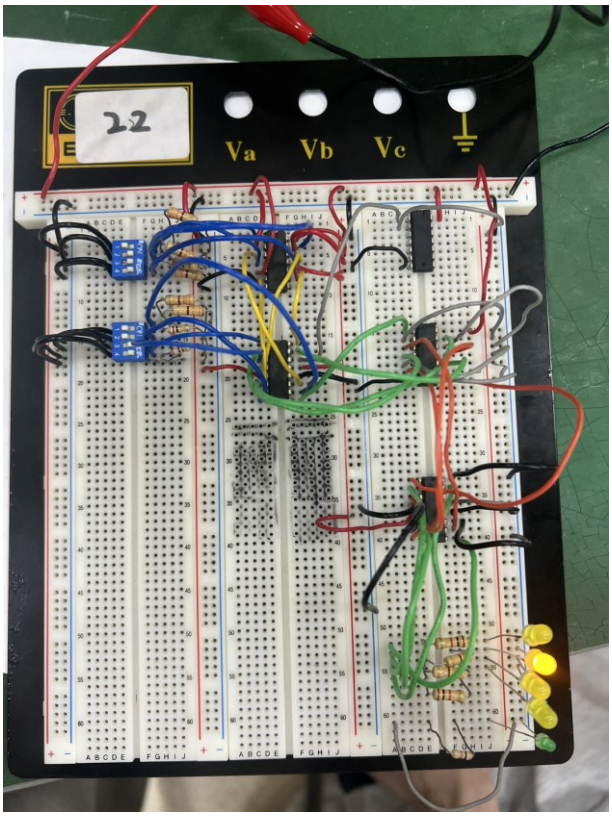
   - 注意：第一個 7483 的 S3-S0 可接至第二個 7483 的 A0-A3 或 B0-B3 ，但接完一側後，**另一側不可不接，需要連接 GND**（我們認為是因為 7483 使用 TTL 技術，TTL 的未接輸入不會默認為 0，而是會被視為不穩定的高電位或受雜訊影響）。

5. **第二個 7483 的 Cin 也接 Cout'**：
   - 如果結果是正數（Cout' = 0），則會 +0，不影響最終結果，S3-S0 即為正數的最終結果。
   - 如果結果是負數（Cout' = 1），則會 +1，使其完成二的補數，S3-S0 即為負數的最終結果。

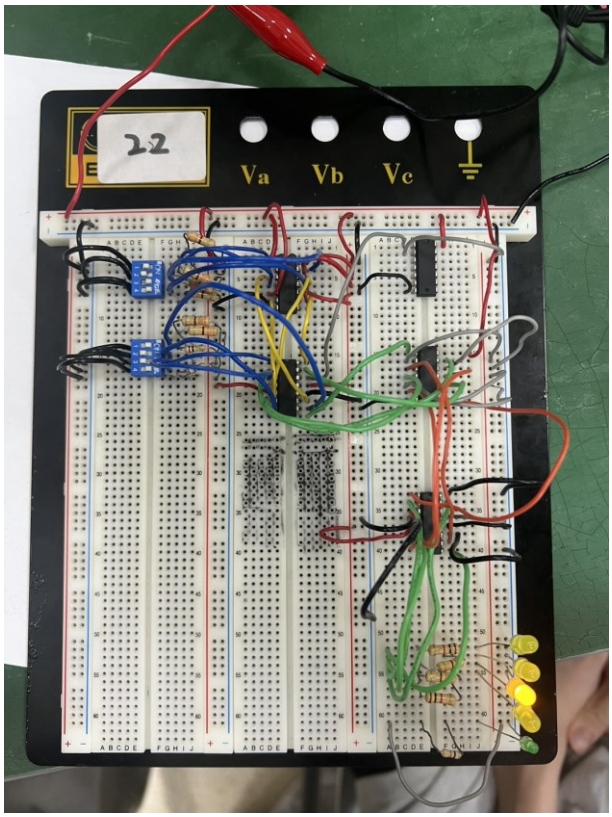
【電路圖】



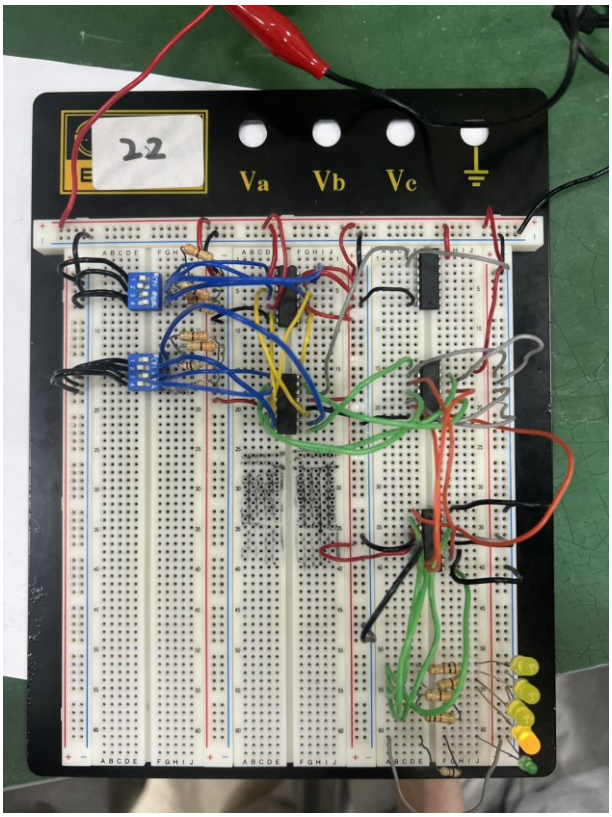
演示：2(0010) - 1(0001) = 1(0001)



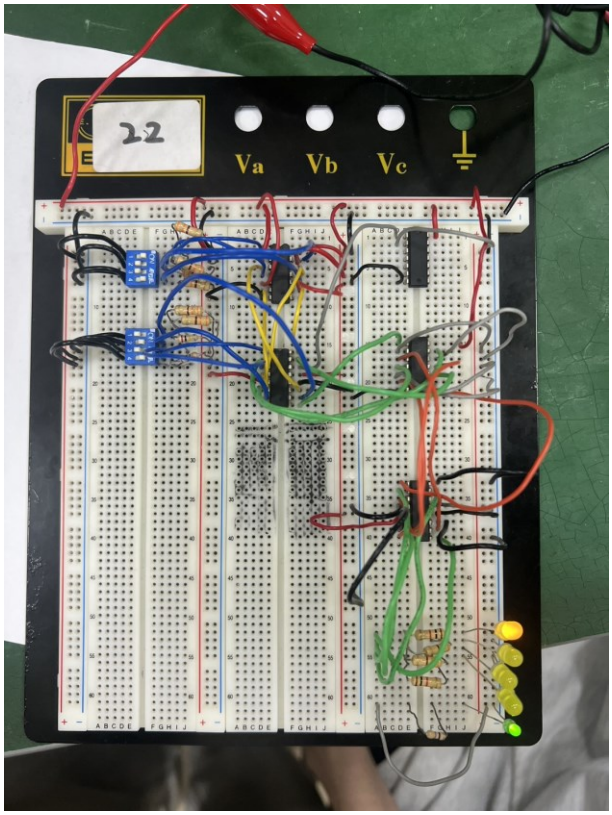
演示：3(0011) - 1(0001) = 2(0010)



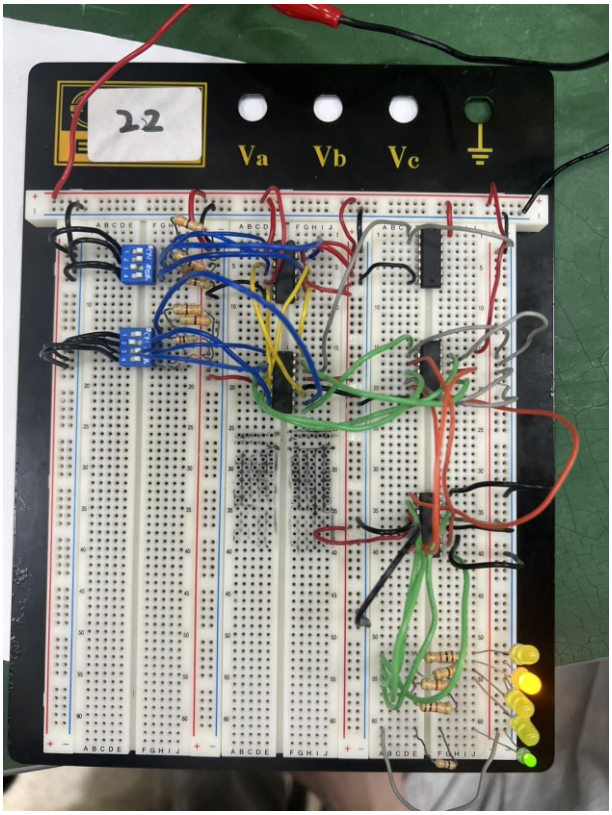
演示：6(0110) - 1(0010) = 4(0100)



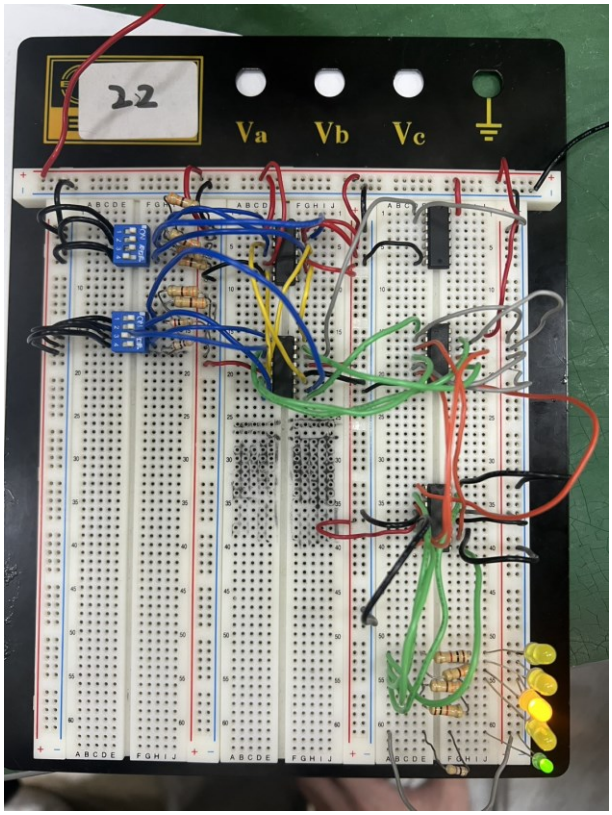
演示：10(1010) - 2(0010) = 8(1000)



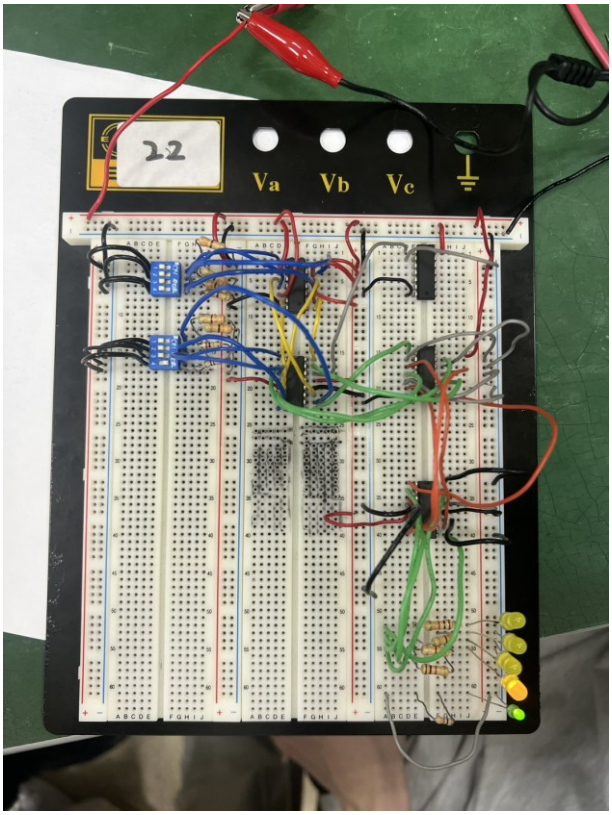
演示：5(0101) - 6(0110) = -1(0001)



演示：2(0010) - 4(0100) = -2(0010)



演示：4(0100) - 8(1000) = -4(0100)



演示：7(0111) - 15(1111) = -8(1000)